

MONGODB – APUNTE

La fortaleza de Mongo reside en su versatilidad, poder, facilidad de uso y habilidad de manejar trabajos pequeños y grandes.

Al igual que HBase y a diferencia de Postgres, no posee un esquema definido, por lo que documentos pueden contener campos o tipos que ningún otro documento posee.

HU(MONGO)US

Mongo es una base de datos basada en documentos de tipo JSON. Puede ser mapeada a una base de datos relacional sin un esquema.

```
{
  "_id" : ObjectId("4d0b6da3bb30773266f39fea"),
  "country" : {
    "$ref" : "countries",
    "$id" : ObjectId("4d0e6074deb8995216a8309e")
  },
  "famous_for" : [
    "beer",
    "food"
  ],
  "last_census" : "Sun Jan 07 2018 00:00:00 GMT -0700 (PDT)",
  "mayor" : {
    "name" : "Ted Wheeler",
    "party" : "D"
  },
  "name" : "Portland",
  "population" : 582000,
  "state" : "OR"
}
```

} JSON document

A diferencia de las bases de datos relacionales, Mongo requiere que se hagan asunciones de cómo se va a usar la data pero pocas con respecto a qué se va a almacenar.}

Gracias a su falta de un esquema estructurado, Mongo puede crecer y cambiar junto con tu modelo de data. Si eres una pequeña startup con sueños de grandeza o una gran corporación que necesita escalar horizontalmente, Mongo es para tí.

DAY 1: CRUD AND NESTING

COMMAND-LINE FUN

Para conectarnos a la interfaz de comandos inspirada en MySQL:

```
$ mongo book
```

Para crear una colección en Mongo añadimos el record inicial a la colección. Como no tiene esquema, no hace falta predefinir muchas cosas:

```
> db.towns.insert({
  name: "New York",
  population: 22200000,
  lastCensus: ISODate("2016-07-01"),
  famousFor: [ "the MOMA", "food", "Derek Jeter" ],
  mayor : {
    name : "Bill de Blasio",
    party : "D"
  }
})
```

Para verificar que la colección existe:

```
> show collections
```

Para listar los contenidos de una colección (el formato del texto puede variar):

```
> db.towns.find()

{
  "_id" : ObjectId("59093bc08c87e2ff4157bd9f"),
  "name" : "New York",
  "population" : 22200000,
  "lastCensus" : ISODate("2016-07-01T00:00:00Z"),
  "famousFor" : [ "the MOMA", "food", "Derek Jeter" ],
  "mayor" : {
    "name" : "Bill de Blasio",
    "party" : "I"
  }
}
```

No soporta joins desde el servidor, una llamada en JavaScript va a devolver un documento y todo su contenido anidado.

Las llaves primarias son siempre de 12 bytes, compuestas de un timestamp, el id de la máquina cliente, el id del proceso de la máquina cliente y un contador de 3 bytes

incremental. Esto evita colisiones con otras instancias de Mongo.

JAVASCRIPT

El lenguaje nativo de Mongo es JavaScript. El lenguaje las bases de datos como un objeto.

```
> typeof db
object
> typeof db.towns
object
> typeof db.towns.insert
function
```

Para insertar en nuestra base de datos:

```
mongo/insertCity.js
function insertCity(
  name, population, lastCensus,
  famousFor, mayorInfo
) {
  db.towns.insert({
    name: name,
    population: population,
    lastCensus: ISODate(lastCensus),
    famousFor: famousFor,
    mayor : mayorInfo
  });
}
```

También se puede pegar el código de la función en el shell, luego, la llamamos:

```
> insertCity("Punxsutawney", 6200, '2016-01-31',
  ["Punxsutawney Phil"], { name : "Richard Alexander" }
)
> insertCity("Portland", 582000, '2016-09-20',
  ["beer", "food", "Portlandia"], { name : "Ted Wheeler", party : "D" }
)
```

READING: MORE FUN IN MONGO

find() también puede recibir parámetros para hacer búsquedas más específicas (hay que usar un wrapper ObjectId(str))

```
> db.towns.find({ "_id" : ObjectId("59094288afbc9350ada6b807") })
{
  "_id" : ObjectId("59094288afbc9350ada6b807"),
  "name" : "Punxsutawney",
  "population" : 6200,
  "lastCensus" : ISODate("2016-01-31T00:00:00Z"),
  "famousFor" : [ "Punxsutawney Phil" ],
  "mayor" : { "name" : "Richard Alexander" }
}
```

También se puede filtrar qué campos se quieren con un segundo parámetro:

```
> db.towns.find({ _id : ObjectId("59094288afbc9350ada6b807") }, { name : 1 })
{
  "_id" : ObjectId("59093e9eafbc9350ada6b803"),
  "name" : "Punxsutawney"
}
```

To retrieve all fields *except* name, set name to 0 (or false or null).

```
> db.towns.find({ _id : ObjectId("59094288afbc9350ada6b807") }, { name : 0 })
{
  "_id" : ObjectId("59093e9eafbc9350ada6b803"),
  "population" : 6200,
  "lastCensus" : ISODate("2016-01-31T00:00:00Z"),
  "famousFor" : [ "Punxsutawney Phil" ]
}
```

También se pueden construir queries ad-hoc basándonos en valores de campo, rangos o una combinación de criterios:

```
> db.towns.find(
  { name : /^P/, population : { $lt : 10000 } },
  { _id: 0, name : 1, population : 1 }
)
{ "name" : "Punxsutawney", "population" : 6200 }
```

Los operadores condicionales en Mongo siguen el formato de field:{\$op : value}, dónde \$op es una operación como \$ne (not equal) o \$gt(greater than):

```

> var population_range = {
  $lt: 1000000,
  $gt: 10000
}
> db.towns.find(
  { name : /^P/, population : population_range },
  { name: 1 }
)
{ "_id" : ObjectId("59094292afbc9350ada6b808"), "name" : "Portland" }

```

También podemos usar operadores como \$gte (greater than or equal) y fechas:

```

> db.towns.find(
  { lastCensus : { $gte : ISODate('2016-06-01') } },
  { _id : 0, name: 1 }
)
{ "name" : "New York" }
{ "name" : "Portland" }

```

DIGGING DEEP

Mongo ama los arrays de información anidados. Podemos consultar matcheando los datos exactos:

```

> db.towns.find(
  { famousFor : 'food' },
  { _id : 0, name : 1, famousFor : 1 }
)
{ "name" : "New York", "famousFor" : [ "the MOMA", "food", "Derek Jeter" ] }
{ "name" : "Portland", "famousFor" : [ "beer", "food", "Portlandia" ] }

```

0 datos parciales:

```

> db.towns.find(
  { famousFor : /moma/ },
  { _id : 0, name : 1, famousFor : 1 }
)
{ "name" : "New York", "famousFor" : [ "the MOMA", "food" ] }

```

0 la falta de datos coincidentes:

```

> db.towns.find(
  { famousFor : { $nin : ['food', 'beer'] } },
  { _id : 0, name : 1, famousFor : 1 }
)
{ "name" : "Punxsutawney", "famousFor" : [ "Punxsutawney Phil" ] }

```

Para listar resultados de subdocumentos profundamente anidados (se usa un punto para separar los distintos niveles de profundidad i.e: mayor.party):

```
> db.towns.find(
  { 'mayor.party' : 'D' },
  { _id : 0, name : 1, mayor : 1 }
)
{ "name" : "New York", "mayor" : { "name" : "Bill de Blasio", "party" : "D" } }
{ "name" : "Portland", "mayor" : { "name" : "Ted Wheeler", "party" : "D" } }
```

or those with mayors who don't have a party:

```
> db.towns.find(
  { 'mayor.party' : { $exists : false } },
  { _id : 0, name : 1, mayor : 1 }
)
{ "name" : "Punxsutawney", "mayor" : { "name" : "Richard Alexander" } }
```

elemMatch

Digamos que tenemos los siguientes países:

```
> db.countries.insert({
  _id : "us",
  name : "United States",
  exports : {
    foods : [
      { name : "bacon", tasty : true },
      { name : "burgers" }
    ]
  }
})
> db.countries.insert({
  _id : "ca",
  name : "Canada",
  exports : {
    foods : [
      { name : "bacon", tasty : false },
      { name : "syrup", tasty : true }
    ]
  }
})
```

```
> db.countries.insert({
  _id : "mx",
  name : "Mexico",
  exports : {
    foods : [{
      name : "salsa",
      tasty : true,
      condiment : true
    }]
  }
})
```

También podemos especificar que nuestra consulta cumpla con todos nuestros criterios con \$elemMatch:

```
> db.countries.find(
  {
    'exports.foods' : {
      $elemMatch : {
        name : 'bacon',
        tasty : true
      }
    }
  },
  { _id : 0, name : 1 }
)
{ "name" : "United States" }
```

También podemos usar operadores avanzados:

```
> db.countries.find(
  {
    'exports.foods' : {
      $elemMatch : {
        tasty : true,
        condiment : { $exists : true }
      }
    }
  },
  { _id : 0, name : 1 }
)
{ "name" : "Mexico" }
```

BOOLEAN OPS

Podemos utilizar operadores booleanos para realizar búsquedas:

```

db.countries.find(
  {
    $or : [
      { _id : "mx" },
      { name : "United States" }
    ]
  },
  { _id:1 }
)
{ "_id" : "us" }
{ "_id" : "mx" }

```

A continuación se deja una lista de los operadores más utilizados:

Command	Description
\$regex	Match by any PCRE-compliant regular expression string (or just use the // delimiters as shown earlier)
\$ne	Not equal to
\$lt	Less than
\$lte	Less than or equal to
\$gt	Greater than
\$gte	Greater than or equal to
\$exists	Check for the existence of a field
\$all	Match all elements in an array
\$in	Match any elements in an array
\$nin	Does not match any elements in an array
\$elemMatch	Match all fields in an array of nested documents
\$or	or
\$nor	Not or
\$size	Match array of given size
\$mod	Modulus
\$type	Match if field is a given datatype
\$not	Negate the given operator check

UPDATING

La cláusula update() recibe 2 parámetros:

- **criteria:** Lo mismo que se le pasaría a un `find()`.
- **operation:** Puede ser:
 - Un objeto cuyos campos van a reemplazar los del documento coincidente
 - Operación modificadora.

En este ejemplo usaremos el modificador “OR”:

```
db.towns.update(
  { _id : ObjectId("4d0ada87bb30773266f39fe5") },
  { $set : { "state" : "OR" } }
);
```

La operación `$set` es necesaria para indicar que lo que queremos hacer es una modificación dentro de un documento. Como Mongo es orientado a documentos, si falta esta operación, éste va a interpretar que queremos reemplazar el documento entero esto es PELIGROSO.

También podemos usar otras operaciones (como `$inc` para incrementar la población de un estado):

```
db.towns.update(
  { _id : ObjectId("4d0ada87bb30773266f39fe5") },
  { $inc : { population : 1000 } }
)
```

A continuación se listan las operaciones más usadas:

Command	Description
<code>\$set</code>	Sets the given field with the given value
<code>\$unset</code>	Removes the field
<code>\$inc</code>	Adds the given field by the given number
<code>\$pop</code>	Removes the last (or first) element from an array
<code>\$push</code>	Adds the value to an array
<code>\$pushAll</code>	Adds all values to an array
<code>\$addToSet</code>	Similar to push, but won't duplicate values
<code>\$pull</code>	Removes matching values from an array
<code>\$pullAll</code>	Removes all matching values from an array

REFERENCES

Aunque no se usen joins, a veces es necesario referenciar otros documentos. En estos casos usamos un constructo de tipo: { \$ref: "collection_name", \$id: "reference_id"}:

```
> db.towns.update(
  { _id : ObjectId("59094292afbc9350ada6b808") },
  { $set : { country: { $ref: "countries", $id: "us" } } }
)
```

Ahora se puede recuperar Portland desde nuestra colección de pueblos:

```
> var portland = db.towns.findOne(
  { _id : ObjectId("59094292afbc9350ada6b808") }
)
```

Luego, se puede recuperar el país al que pertenece, podemos consultar a la colección de países:

```
> db.countries.findOne({ _id: portland.country.$id })
```

También podemos guardar esta información en una referencia:

```
> var portlandCountryRef = portland.country.$ref;
> db[portlandCountryRef].findOne({ _id: portland.country.$id })
```

DELETING

Funciona de manera similar al find(), se recomienda primero correr find() para asegurarse qué es lo que se está eliminando.

```

> var badBacon = {
  'exports.foods' : {
    $elemMatch : {
      name : 'bacon',
      tasty : false
    }
  }
}
> db.countries.find(badBacon)
{
  "_id" : ObjectId("4d0b7b84bb30773266f39fef"),
  "name" : "Canada",
  "exports" : {
    "foods" : [
      {
        "name" : "bacon",
        "tasty" : false
      },
      {
        "name" : "syrup",
        "tasty" : true
      }
    ]
  }
}

```

Todo se ve bien, ahora a eliminarlo:

```

> db.countries.remove(badBacon)
> db.countries.count()
2

```

READING WITH CODE

Se pueden crear funciones de decisión. Esto debe ser la última opción. Son muy lentas, no se pueden indexar y, además, Mongo no las puede optimizar, aunque a veces la lógica del código personalizado es difícil de superar.

```

> db.towns.find(function() {
  return this.population > 6000 && this.population < 600000;
})

```

Una forma más sencilla:

```
> db.towns.find("this.population > 6000 && this.population < 600000")
```

La cláusula \$where nos permite implementar otros criterios:

```
db.towns.find({  
  $where: "this.population > 6000 && this.population < 600000",  
  famousFor: /Phil/  
})
```